Available online at www.sciencedirect.com

ScienceDirect

Transportation Research Procedia 86 (2025) 337–344

Transportation Research Procedia

www.elsevier.com/locate/procedia

26th Euro Working Group on Transportation Meeting (EWGT 2024)

Late Train-Crew Rescheduling—a Model and a Heuristic Approach

Liyun Yu[a,*], Carl Henrik Häll[a], Anders Peterson[a], Christiane Schmidt[a]

[a]*Department of Science and Technology, Linköping University, Norrköping, Sweden*

Abstract

In this paper, we reschedule the duties of train drivers one day before the operation. Due to absent drivers (e.g., because of sick leave), some trains have no driver. Thus, duties on the operation day need to be rescheduled. We start with a feasible crew schedule of the remaining operating drivers, a set of unassigned tasks which were assigned to the absent drivers, and a group of standby drivers with fixed start time, end time, start depot, and end depot. Our aim is to generate a crew schedule with as few cancelled driving tasks and changed tasks as possible. We developed a Mixed Integer Linear Programming (MILP) model for short-term crew rescheduling. We also present a Tabu-Search-Based approach with the same objective function and equivalent restrictions of our mathematical model. We compare the results of the MILP model solved by Gurobi 11.0 and of the Tabu-Search-Based approach. Our Tabu-Search-Based approach takes less time and memory occupation to compute a 'good-enough' result. We further test the performance of our approach under different circumstances. The data used in the experiments are real data provided by Mälartåg.

© 2024 The Authors. Published by ELSEVIER B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the 26th Euro Working Group on Transportation Meeting

Keywords: Railway crew rescheduling; Mixed integer linear programming; Tabu search

1. Introduction

Over decades, railway disruption has been an important topic, and canceling trains is one of the common, but not most preferred, options. However, passengers still face possible severe delays and cancellations of trains during traveling. The causes could be last changes in the railway schedule including timetable, rolling stock, and crew schedules. For society, it is important to maintain the railway schedule as planned, instead of having too many changes on short notice. The rail industry wants to cancel as few trains as possible during rescheduling since the cancellation cost is high. The cancellation cost includes both financial loss and employees' working time loss since they need to spend extra time and energy to move the canceled train back to a specific depot. Moreover, trains can be canceled for various reasons, such as derailment, extreme weather, and resource allocation. Handling a lack of train crew is a type of resource reallocation. In this paper, we explore short-term crew rescheduling and try to cancel as few trains as possible.

* Corresponding author.
E-mail address: liyun.yu@liu.se

2352-1465 © 2024 The Authors. Published by ELSEVIER B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the 26th Euro Working Group on Transportation Meeting
10.1016/j.trpro.2025.04.043

We consider the case when several train drivers take leave for the next day and the rail undertaking needs to generate a new schedule for the remaining on-duty drivers and some standby drivers starting from different depots. Cancellations of trains are still considered, but not preferred. In our case, we have a group of drivers who have a feasible schedule in the original timetable, some unassigned tasks, and standby drivers without any assigned tasks. Initially, we have feasible schedules with a one-day length for all on-duty drivers, including standby drivers and drivers with assigned tasks. We assume that there are a certain number of drivers who take leaves at short notice. Thus, we need to assign the tasks, which were assigned to the absent drivers, to other remaining on-duty drivers. Our goal is to propose a Tabu-Search-based approach and a MILP model with equivalent limitations for achieving feasible solutions by rescheduling the drivers' tasks under different objective functions, e.g., the minimum number of train cancellations. We compare the computational time and space between our Tabu-Search-based approach and commercial solvers with the MILP model. Apart from considering the legal constraints regarding the employees' rights, we also consider possible constraints regarding the rail undertakings' and their employees' preferences. We limit ourselves to generating a one-day crew schedule and only consider deadheading by the trains existing in the original schedule, that is, no trains of other rail undertakings (RUs) can be used for deadheading.

This paper is organized as follows. We present the related work in Section 2. In Section 3 and 4, we show the mathematical model and Tabu-Search-based approach, respectively. Then, we run experiments and analyse the results in Section 5 and conclude the paper in Section 6.

2. Related Work

Railway scheduling mainly involves the steps of train timetabling, rolling stock scheduling, and crew planning (Cacchiani et al., 2014; Heil et al., 2020). Heil et al. (2020) detailed that crew planning includes two steps, which are crew scheduling and crew rostering. Crew scheduling is to form duties with tasks and crew rostering is to combine duties into weekly schedules and assign them to different crews (Jütte et al., 2011). Abbink et al. (2018) mentioned that a duty usually covers a short time, for example, one day. A task contains several elements, including the start time, start depot, end time, and end depot. According to Heil et al. (2020), crew rescheduling includes both short-term (one-day schedule) and long-term (schedules of more than a day) rescheduling.

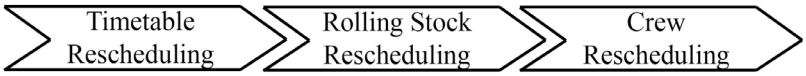


Fig. 1. Rescheduling Process Flow

The common rescheduling process in railway starts with rescheduling the timetable followed by rolling stock rescheduling, and then ends with crew rescheduling, see Figure 1. To the best of our knowledge, most of the researchers consider crew rescheduling as long-term rescheduling (Heil et al., 2020; Huisman, 2007), or as short-term rescheduling but the crew schedules are affected by changes in railway timetables (Potthoff et al., 2010; Verhaegh et al., 2017; Abbink et al., 2018; Yuan et al., 2022). Cacchiani et al. (2014) mentioned that although the set-cover formulations are commonly used for crew rescheduling, they are not suitable for real-time rescheduling due to the long computation times.

There are several common constraint types for the schedules. The basic ones include the geographical and chronological connection between consecutive tasks and the location restrictions imposed by the start and end depots of all drivers. Other time-related constraints can be classified in two main categories. The first one relates to the working time, including total working time and uninterrupted working time. The total working time restricts the total working time for each driver including work, deadheading, transit and break time (Boschetti et al., 2004; Kokubo and Fukuyama, 2017a). The working time gives an upper time limit of consecutive working without a break. Chen and Shen (2013) considered forcing to assign a break between two consecutive tasks and limits the maximum length of each duty. Veelenturf et al. (2016) limited the length of the working time before and after a break separately. Fuentes et al. (2019) directly limited the maximum continuous driving time without any break. The second category of time-related constraints is about breaks, including the number and duration of breaks and the starting time range of breaks. The number and duration of breaks for each duty should be limited.Verhaegh et al. (2017) explored different break durations and their effects. Kokubo and Fukuyama (2017a) limited the break duration by giving a time range.

Due to the necessarily large number of binary variables, we decide to explore the possibilities of heuristic approaches, which are the most common option in crew rescheduling, especially, the approaches considering column

generation (Potthoff et al., 2010; Veelenturf et al., 2012). However, Heil et al. (2020) pointed out that the computational time of column generation can be very long due to problem size and slow convergence. Among meta-heuristics approaches, Heil et al. (2020) mentioned that genetic algorithms cannot maintain the feasibility of schedules in all iterations. Since our problem contains several remaining individual drivers with feasible schedules, we use Tabu Search. There are some studies of using Tabu Search in crew scheduling (Elizondo et al., 2010; Chew et al., 2001; Kokubo and Fukuyama, 2018). Chew et al. (2001) used a Tabu-Search algorithm for their set partitioning formulation for crew scheduling. They introduced three move operators including exchanging one task in each duty, exchanging a sequences of tasks in each duty, and moving one task to another duty. Kokubo and Fukuyama (2017b) introduced a Tabu-Search-based approach with three types of schedule movements for generating neighborhood schedules. Kokubo and Fukuyama (2017a) added three more categories to make the Tabu Search more powerful when scheduling. Kokubo and Fukuyama (2018) changed their approach into parallel Tabu Search. There is also some research of small-scale crew rescheduling solved by other approaches with different operations to generate possible neighbouring schedules. Yuan et al. (2022) considered directly inserting the unassigned task or swapping it with assigned tasks. Verhaegh et al. (2017) introduced an iterative-deepening depth-first search for rescheduling. They considered the operation of either inserting one selected task or swapping the selected task and conflicted assigned tasks by considering deadheading at most two consecutive tasks. However, in most studies, crew rescheduling becomes necessary because of changes in the timetable.

The main difference between our work and other studies on crew rescheduling is that we start with feasible driver-task assignments for a subset of drivers and tasks and that only drivers' schedules (but not the timetable) are impacted. In our case, we have a group of drivers who has a feasible schedule in the original timetable and a set of unassigned tasks. While in other studies, researchers need to significantly change the original timetable due to external causes. The original crew schedule for most of the drivers becomes infeasible. Thus, they do not have many drivers with a feasible schedule at the beginning. Our main contributions can be classified into two parts. The first one is a MILP model. We formulate a MILP model and consider more aspects comparing to previous studies, besides total working hour, breaks and driver's license limits. For example, we consider drivers' preference for overwork and the rail undertakings' preferences for the new schedule. Moreover, we introduce a Tabu-Search-based approach with equivalent limitations and more types of operations to generate neighborhood solutions comparing to previous studies.

3. Mathematical Model

In this section, we present our MILP model of short-term crew rescheduling. Table 1 shows the indexes and sets used in our model. We display the variables and parameters separately in Table 2 and 3.

Table 1. Indexes and Sets

Indexes and Sets	Description
g	Task
d	Driver
G	The set of all task, where $g \in G$
D	The set of all available drivers in the new schedule
$\hat{D}$	The set of originally scheduled drivers in the old schedule
D^u	The set with only one element, which is a shadow driver for the unassigned tasks

Table 2. Variables

Variables	Set Description
$x_{g,d}$	Binary variable, which equals to 1 if task g is included in the duty of driver d
$b_{g,d}$	Binary variable, which equals to 1 if a break starts right after task g for driver d
$z_{g,d}$	Continuous variable, which is forced to vary either 0 or 1. It indicates whether the new schedule for task g to driver d has been changed in the new schedule

Table 3. Parameters

Parameters	Description
λ	A constant in $[0, 1]$
M	Big M
t^{b}	Minimum break time duration
t^{w}	Maximum working hour limit
t^{wo}	Maximum work hour without a scheduled break
t_g^{s}	The starting time of task g
t_g^{e}	The ending time of task g
t_d^{es}	The earliest starting time for driver d
t_d^{le}	The latest ending time for driver d
$q_{g,g'}$	Binary parameter, which equals to 1 if tasks g and $g^{'}$ can be assigned to one driver
$p_{g,g'}$	Binary parameter, which equals to 1 if task g can be geographically connected with $g^{'}$ and g occurs before $g^{'}$
$o_{g,g'}$	Binary parameter, which equals to 1 if task g occurs before task $g^{'}$
$r_{g,d}$	Binary parameter, which equals to 1 if task g can be assigned to driver d
$h_{g,d}$	Binary parameter, which equals to 1 if task g can be assigned as the first task of driver d
$t_{g,d}$	Binary parameter, which equals to 1 if task g can be assigned as the last task of driver d
$\hat{x}_{g,d}$	Binary parameter, which equals to 1 if task g is assigned to driver d in the original schedule

$$\text{Minimize} \quad \lambda \sum_{g \in G} \sum_{d \in D \cap \hat{D}} z_{g,d} + (1-\lambda) \sum_{d \in D^u} x_{g,d}\,, \qquad \text{where } \lambda \in [0,1] \quad (1)$$

$$\sum_{d \in D \cup D^u} x_{g,d} \geq 1 \qquad \forall g \in G \quad (2)$$

$$\sum_{d \in D} x_{g,d} \leq M(1 - x_{g,d^*}) \qquad \forall g \in G, d^* \in D^u \quad (3)$$

$$q_{g,g'} + 1 \geq x_{g,d} + x_{g',d} \qquad \forall g \in G, g^{'} \in G, d \in D : g \neq g^{'} \quad (4)$$

$$t_{g'}^{\mathrm{e}} - t_g^{\mathrm{s}} <= t^{\mathrm{w}} + M(2 - x_{g,d} - x_{g',d}) \qquad \forall g \in G, g^{'} \in G, d \in D : g \neq g^{'} \quad (5)$$

$$r_{g,d} \geq x_{g,d} \qquad \forall g \in G, d \in D \quad (6)$$

$$(t_g^{\mathrm{s}} - t_d^{\mathrm{es}}) x_{g,d} \geq 0 \qquad \forall g \in G, d \in D \quad (7)$$

$$(t_d^{\mathrm{le}} - t_g^{\mathrm{e}}) x_{g,d} \geq 0 \qquad \forall g \in G, d \in D \quad (8)$$

$$\sum_{g^* \in G : g \neq g^*} (o_{g,g^*} - o_{g',g^*}) x_{g^*,d} \geq x_{g,d} + x_{g',d} - p_{g,g'} - 1 \qquad \forall g \in G, g^{'} \in G, d \in D : o_{g,g'} = 1, g \neq g^{'} \quad (9)$$

$$\sum_{g^* \in G : g^* \neq g} o_{g^*,g} x_{g^*,d} \geq 1 - M(1 - x_{g,d}) \qquad \forall g \in G, d \in D : h_{g,d} = 0 \quad (10)$$

$$\sum_{g^* \in G : g^* \neq g} o_{g,g^*} x_{g^*,d} \geq 1 - M(1 - x_{g,d}) \qquad \forall g \in G, d \in D : t_{g,d} = 0 \quad (11)$$

$$b_{g,d} \leq x_{g,d} \qquad \forall g \in G, d \in D \quad (12)$$

$$t_{g'}^{\mathrm{s}} - t_g^{\mathrm{e}} \geq t^{\mathrm{b}} - M \cdot (1 - o_{g,g'}(b_{g,d} + x_{g',d} - 1)) \qquad \forall g \in G, g^{'} \in G, d \in D : g \neq g^{'} \quad (13)$$

$$\sum_{g \in G} b_{g,d} \leq 1 \qquad \forall d \in D \quad (14)$$

$$\sum_{\substack{g^* \in G: \\ o_{g^*,g'}=1, \\ o_{g,g^*}=1, \\ g^* \neq g'}} b_{g^*,d} \geq x_{g',d} + x_{g,d} - 1 \qquad \forall g \in G, g' \in G, d \in D : t^{e}_{g'} - t^{s}_{g} \geq t^{wo} \quad (15)$$

$$z_{g,d} \geq x_{g,d} - \hat{x}_{g,d} \qquad \forall g \in G, d \in D \cap \hat{D} \quad (16)$$

$$z_{g,d} \geq -x_{g,d} + \hat{x}_{g,d} \qquad \forall g \in G, d \in D \cap \hat{D} \quad (17)$$

$$z_{g,d} \geq 0 \qquad \forall g \in G, d \in D \cap \hat{D} \quad (18)$$

$$x_{g,d} \in \{0,1\} \qquad \forall g \in G, d \in D \cup D^{u} \quad (19)$$

$$b_{g,d} \in \{0,1\} \qquad \forall g \in G, d \in D \quad (20)$$

We aim to minimize a convex combination of the total number of changes from the original schedule and the total number of unassigned tasks at the same time. Constraint (2) enforces that each task is either assigned to some driver or to the pool of unassigned tasks. If task g is assigned to multiple drivers, then we consider the task g for all but one driver as a deadheading task. Constraint (3) limits the relation between assigning one task to all drivers and assigning the same task to the unassigned task pool. If the task g is in the unassigned pool, which means $x_{g,d^*} = 1$, where $d^* \in D^u$, then task g cannot be assigned to any driver $d \in D$, and vice versa. Constraint (4) is used to avoid time conflicts among tasks within one driver's duty. All the tasks assigned to one driver's duty should not have time conflicts. Thus, we can only assign two tasks g and g' together to driver d if parameter $q_{g,g'} = 1$. Constraint (5) limits the total working time of each driver d. We check for any two tasks g and g' assigned to driver d whether $t^{e}_{g'} - t^{s}_{g}$ exceeds the maximum total working hour limit for each driver. Constraint (6) restricts the type of vehicles that each driver can drive based on their licenses. If the driver d is allowed to operate the vehicle type of task g, then the task g is allowed to be assigned to driver d. We also limit the earliest start and latest end time of drivers in Constraints (7) and (8). Any task g assigned to driver d must satisfy that task g starts later than the earliest starting time of driver d and ends earlier than the latest ending time of driver d. These two constraints are set to fulfill the minimum rest hour rule between two consecutive shifts. Constraint (9) makes sure that any two consecutive tasks g and g' assigned to driver d must satisfy depot consistency. If task g and g' are two consecutive tasks assigned to driver d and task g occurs before g', then the ending depot of task g and the starting depot of g' should be the same. Constraints (10) and (11) restrict the depot location of the first and last task assigned to each driver. The starting depot of the first task g assigned to driver d should be this driver's home depot, enforced by Constraint (10). In Constraint (11), the ending depot of the last task g assigned to driver d should also be this driver's home depot. Constraints (12)-(15) are limitations regarding breaks. Constraint (12) indicates that driver d can only have a break right after task g if task g is assigned to driver d. Constraint (13) gives a lower bound on the break-time duration of each driver. Constraint (14) sets the maximum number of breaks for each driver's shift to 1. Constraint (15) restricts the maximum working time without scheduling any break in the driver's shift. Constraints (16) - (18) make sure that the continuous variable $z_{g,d}$ equals to the absolute value of $x_{g,d} - \hat{x}_{g,d}$ for each task g and each driver d. Constraints (19) and (20) indicate that variable $x_{g,d}$ and $b_{g,d}$ are binary variables. The proposed mathematical model contains $O(|G|\,|D|)$ variables with $|G|\,|D|$ continuous variables and $2*|G|\,|D|+|G|$ binary variables. The constraints number in this model is $O(|G|^2\,|D|)$, and it can be $4.5*|G|^2\,|D| + 7.5*|G|\,|D| + 3\,|G| + |D|$ at worst.

4. Tabu-Search-Based Approach

We use a Tabu-Search-based approach. In Algorithm 1, we show the overall structure of this approach. The approach starts with the initial schedule s_0, including the remaining drivers with feasible schedules, a set of unassigned tasks that were assigned to the absent drivers originally, and the set of standby drivers with the start and end working time and depot location. We consider the set of unassigned tasks assigned to the absent drivers in the original schedule

as the unassigned task pool. Then we randomly select one unassigned task. After that, we follow the structure of Tabu Search. To determine the best schedule among neighbouring solutions, we find the one with the minimum number of unassigned tasks. For the Tabu Tenure, we track the iteration number of each schedule in the Tabu List. If the iteration number of any stored schedule exceeds the maximum limit, then we release it from the Tabu List. We use three termination criteria. The first one is the limitation of the maximum number of iterations. The second one depends on whether the set of unassigned task is empty or not. The last one limits the maximum number of times a tasks can be tried to be assigned to any driver. If all the unassigned tasks left have already failed a certain number of time, then the approach should also be terminated.

Algorithm 1 Tabu-Search-based approach

Require: A feasible schedule for all remaining on-duty drivers s_0 & A set of unassigned tasks
Ensure: Historical best solution s^*
Initialization: tabu list $T \leftarrow null$, best neighbouring solution $s' \leftarrow null$, current solution $s \leftarrow$ initial solution s_0, historical best solution $s^* \leftarrow$initial solution s_0
for $i = 1$ **to** $MaxIterNum$ **do**
  Select an unassigned task g
  Generate neighbouring solutions $N(s)$ of the unassigned task
  Find the best neighbouring solution s' in $N(s)$
  if best neighbouring solution s' better than historical best solution s^* **then**
    historical best solution $s^* \leftarrow$ best neighbouring solution s'
  end if
  current solution $s \leftarrow$ historical best solution s^*
  Update tabu list T based on tabu tenure
  if any termination criteria satisfied **then**
    break
  else
    continue
  end if
end for

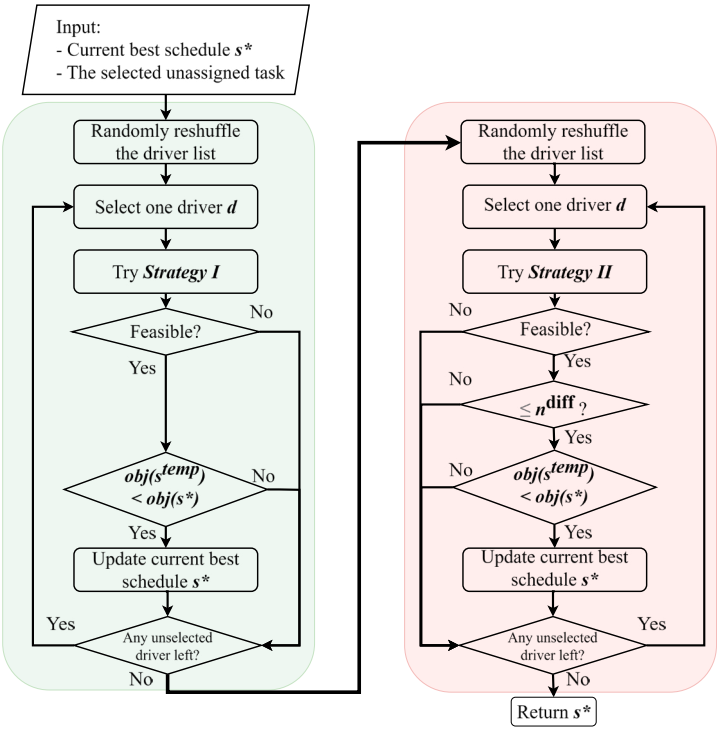


Fig. 2. Generate neighbouring solutions

Figure 2 displays the process flow of generating neighbouring solutions and determining the best solution among all the possible options. At the beginning, we start with the given selected task and the current best schedule s^*. While exploring, we aim to maintain the feasibility of the neighbourhood solution s^{temp} through feasibility checks. Parameter n^{diff} refers to the maximum allowed difference between the number of tasks unassigned from driver d and the number of tasks assigned to driver d. This parameter is a tool for restricting how much level of freedom the approach can have while searching neighbourhood solutions. To determine whether the found neighbourhood solution is better than the initial one, we compare their objective function values. The objective function is the same objective function as for our mathematical model, see Function (1). While generating the neighbouring solutions, we mainly include two strategies. For Strategy I, we only consider assigning tasks to the drivers without unassigning any tasks. For Strategy II, we try to assign tasks to the drivers and delete some tasks from this driver's schedule correspondingly. In both strategies, we consider to add some extra tasks both before and after the selected unassigned task to maintain the consistency of depot location. In our strategies, we consider different options of assigning extra tasks including deadheading with only 1 task, deadheading with several continuous tasks, and assign another unassigned task. Both Strategy I and II maintain the feasibility of the neighbourhood solution from the aspects of depot consistency and time. The time aspect relates with Constraint (4), while the depot aspect relates with Constraints (9) - (11). Overall, we consider the same restrictions as in the mathematical constraints presented in Section 3. For example, we avoid time conflicts between tasks of one driver's duty, which is equivalent to Constraint 4.

5. Case Study

In this section, we compare the results from the mathematical model solved by Gurobi 11.0 and the Tabu-Search-based approach with smaller-size schedules. The goal is to investigate whether our approach can achieve a 'good enough' schedule. The data used in this section is from Mälartåg in Sweden. There are 11 stations considered in our case study: Arboga, Eskiltuna, Gävle, Hallsberg, Linköping, Norrköping, Nyköping, Sala, Stockholm, Uppsala, and Västerås. We solve our mathematical model with Gurobi 11.0 on Tetralith, which is a general computational resource hosted by NSC at Linköping University. We run the Tabu-Search-based approach on a computer with AMD Ryzen 5 PRO 4650U and 16.0GB RAM.

Table 4. Data size

Data size	#Driver with duties	#Standby driver	#Absent driver	#Unassigned Tasks
small	50	8	4	18
medium	74	12	6	22
large	99	16	9	37

Table 5. Computational time, memory occupation, and optimality results

Data size	Method	Time	Memory Occupation	Assignment Rate
small	Tabu-Search-based Approach	9.7 s	0.16 GB	8/18
	MILP Model (Gurobi 11.0)	0.7 h	12.24 GB	10/18
medium	Tabu-Search-based Approach	12.3 s	0.18 GB	12/22
	MILP Model (Gurobi 11.0)	7.5 h	35.00 GB	17/22
large	Tabu-Search-based Approach	21.7 s	0.20 GB	29/37
	MILP Model (Gurobi 11.0)	-	out of memory	-

We have three schedules with different sizes, which are listed in Table 4. For example, in the medium-sized schedule, we have 74 drivers with assigned tasks and 12 standby drivers initially. We assume that there are 6 absent ones among those 74 drivers, and the absent drivers have 22 tasks in the initial schedule. Furthermore, the largest instance is a complete one-day schedule. Table 5 shows the computational times, the memory occupation in gigabytes (GB), and assignment rates of both the Tabu-Search-based approach and the MILP model solved by Gurobi in the initial schedules with different numbers of drivers. In the test run of the small schedule, the Tabu-Searched-based approach only used 9.7 seconds and 0.16 GB to achieve a result, while the MILP model takes 0.7 hours and 12.24 GB. For the medium schedule, the Tabu-Searched-based approach only takes 12.3 seconds to output a result, while Gurobi, for solving the MILP model, takes around 7.5 hours to achieve the optimum for the same problem. Also, the Model uses 35.00 GB of memory while the heuristic approach only uses 0.18 GB. Furthermore, the server runs out of memory when we run the model with the large schedule. In Table 5, we show that the issues we have when running the MILP model include not only the computational time but also the memory occupation. The long computational time is an issue since we are close to operation while rescheduling. Furthermore, the issue of running out of memory is also severe.

6. Conclusions

In this paper, we consider the short-term crew rescheduling problem, where only one-day schedules are considered. We formulated a Mixed Integer Linear Programming (MILP) model solved by Gurobi 11.0 and a Tabu-Search-based approach to reschedule train drivers' tasks one day before the operation. Both MILP model and Tabu-Search-based approach share the same objective function and restrictions, including operational limits (time conflicts and depot consistency), total working hour, drivers' license limit, rest, and breaks. Furthermore, we developed different options of exploring neighbourhood solutions by considering deadheading one task, deadheading a group of consecutive tasks, and extra assigning another unassigned task at different places. In the result, we show that our Tabu-Search-based approach outperforms the MILP model in both computational time and memory occupation for different instance sizes. Furthermore, the assignment rate of the Tabu-Search-based approach shows that the result can be considered as 'good enough' by comparing the rate to the optimal result from the MILP model.

For future work, there are mainly two aspects to increase the assignment rate of the approach. Firstly, we may consider the option of taking taxis for the drivers apart from deadheading. Secondly, we aim to explore more possible options while finding neighboring solutions.

Acknowledgements

Thanks to the discussion and data from Mälartåg including Olov Lindfeldt, Lars Swahn, Freddy Ahlström, and Viktor Reineck. The computational experiments were enabled by resources provided by the National Academic Infrastructure for Supercomputing in Sweden (NAISS), partially funded by the Swedish Research Council through grant agreement no. 2022-06725.

References

Abbink, E., Huisman, D., Kroon, L., 2018. Railway crew management. Handbook of optimization in the railway industry , 243–264.

Boschetti, M.A., Mingozzi, A., Ricciardelli, S., et al., 2004. An exact algorithm for the simplified multiple depot crew scheduling problem. Annals of Operations Research 127, 177–201.

Cacchiani, V., Huisman, D., Kidd, M., Kroon, L., Toth, P., Veelenturf, L., Wagenaar, J., 2014. An overview of recovery models and algorithms for real-time railway rescheduling. Transportation Research Part B: Methodological 63, 15–37.

Chen, S., Shen, Y., 2013. An improved column generation algorithm for crew scheduling problems. Journal of Information and Computational Science 10, 175–183.

Chew, K.L., Pang, J., Liu, Q., Ou, J., Teo, C.P., et al., 2001. An optimization based approach to the train operator scheduling problem at singapore mrt. Annals of operations Research 108, 111–122.

Elizondo, R., Parada, V., Rojas, L., Artigues, C., 2010. An evolutionary and constructive approach to a crew scheduling problem in underground passenger transport. Journal of Heuristics 16. doi:10.1007/s10732-009-9102-x.

Fuentes, M., Cadarso, L., Marín, Á., 2019. A hybrid model for crew scheduling in rail rapid transit networks. Transportation Research Part B: Methodological 125, 248–265.

Heil, J., Hoffmann, K., Buscher, U., 2020. Railway crew scheduling: Models, methods and applications. European journal of Operational Research 283, 405–425.

Huisman, D., 2007. A column generation approach for the rail crew re-scheduling problem. European Journal of Operational Research 180, 163–173.

Jütte, S., Albers, M., Thonemann, U.W., Haase, K., 2011. Optimizing railway crew scheduling at db schenker. Interfaces 41, 109–122.

Kokubo, T., Fukuyama, Y., 2017a. Generation methods of neighborhood schedules for practical train crew scheduling problems using tabu search, in: 2017 IEEE 10th International Workshop on Computational Intelligence and Applications (IWCIA), IEEE. pp. 39–44.

Kokubo, T., Fukuyama, Y., 2017b. Train crew scheduling using tabu search, pp. 63–66. doi:10.23919/SICE.2017.8105640.

Kokubo, T., Fukuyama, Y., 2018. Practical train crew scheduling problems using parallel tabu search, pp. 1673–1678. doi:10.23919/SICE.2018.8492563.

Potthoff, D., Huisman, D., Desaulniers, G., 2010. Column generation with dynamic duty selection for railway crew rescheduling. Transportation Science 44, 493–505.

Veelenturf, L.P., Potthoff, D., Huisman, D., Kroon, L.G., 2012. Railway crew rescheduling with retiming. Transportation research part C: emerging technologies 20, 95–110.

Veelenturf, L.P., Potthoff, D., Huisman, D., Kroon, L.G., Maróti, G., Wagelmans, A.P., 2016. A quasi-robust optimization approach for crew rescheduling. Transportation Science 50, 204–215.

Verhaegh, T., Huisman, D., Fioole, P.J., Vera, J.C., 2017. A heuristic for real-time crew rescheduling during small disruptions. Public Transport 9, 325–342.

Yuan, J., Jones, D., Nicholson, G., 2022. Flexible real-time railway crew rescheduling using depth-first search. Journal of Rail Transport Planning Management 24, 100353. URL: https://www.sciencedirect.com/science/article/pii/S2210970622000531, doi:https://doi.org/10.1016/j.jrtpm.2022.100353.